

Inteligencia de negocios para proyecto PowerBi

Joseph Santiago Jimenez Jimenez

Miguel Ángel Valencia Vélez

Institución Universitaria Pascual Bravo

Facultad de Ingeniería

Ingeniería del Software

INTELIGENCIA DE NEGOCIOS BI

GRUPO 01

Ramiro Grisales Montoya

Octubre 5 de 2026

Bloque 1 · Power BI y Grafana

D1 .¿Qué es Grafana?

Es una plataforma de código abierto similar a Power Bi que permite visualizar, consultar y alertar sobre datos, generando reportes y dashboard, siendo su enfoque principal la observabilidad. Podemos definir los componentes solicitados así:

- Data Sources: Son las fuentes de datos que se conectan a Grafana; la plataforma cuenta con varios plugins para permitirle interactuar con los diferentes programas. Pueden haber fuentes de datos combinados con el origen de tipo *Mixed*
- Paneles: Estas son las unidades básicas de visualización en Grafana, siendo los bloques de construcción de los reportes. Estos constan a su vez de tres partes
    - Tipo: El gráfico que representa la data en el panel (barras, tablas, estadística)
    - Umbrales: Que permiten señalar los datos en estado de interés
    - Trato de datos: En lo que se incluye los datos que pide el panel, las transformaciones y las opciones de visualización
- Dashboards: Son las colecciones de paneles organizadas en una sola página
- Variables: Son marcadores de posición dentro del dashboard, que permiten que este sea dinámico y que pueda cambiar de parámetro.
- Alerting: Es un sistema que evalúa las condiciones de los datos y notifica a un encargado cuando estos presentan algún comportamiento que se haya definido previamente como de interés.

Respecto a Grafana OSS y Cloud, tenemos:

	OSS	Cloud
Modelo de uso	Open source	SaaS hosteado por Grafana Labs
Instalación	Autogestionada dentro de un servidor propio	Se usa desde los sistema de Grafana Labs
Almacenamiento de los datos	Se usan los servicios propios para manejar el almacenamiento y los backups	Tiene sistemas propios manejados por la compañía matriz
Costo	Gratuito	Capa gratuita con planes de us

D2. Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Inteligencia de negocio	Observabilidad y monitoreo de métricas y series de tiempo
Usuario típico	Analista de negocio, gerencia	Ingeniero de operación, DevOps, mantenimiento
Conexión a los datos	Más de 100 conectores	Plugins por fuente
Modelo semántico	Modelo propio con relaciones y medidas	No tiene; cada panel tiene su consulta
Lenguaje	DAX (medidas) y Power Query (M)	El de la fuente más macros
Actualización y tiempo real	Actualización programada; tiempo real limitado	Refresh automático
Alertas	Solo sobre mosaicos del dashboard	Reglas con consulta, condiciones y *contact points*
Licenciamiento y costo	Desktop gratis	Código abierto gratis (autoalojado); Cloud de pago
Control de acceso	RLS con roles y filtros DAX; espacios de trabajo	Organizaciones, equipos y permisos por carpeta o dashboard

D3. Tipos de dashboard y KPI

Primero entendamos qué es cada tipo de dashboard:

- Operativo: Este es un dashboard enfocado en responder qué está pasando en el proceso de interés y qué parte de este requiere acción. Tiene una granularidad fina (de entre 1 y 5 minutos) siendo su enfoque transmitir datos de interés inmediato, como alertas y puntos críticos de la operación. Suelen tener pocos paneles.
- Analítico: Mira hacia eventos pasados principalmente, buscando recopilar datos de eventos sucedidos durante un proceso en una ventana más grande (que puede abarcar días, meses o años) y tiene una granularidad variable, permitiendo hacer drill down a las

métricas para mayor precisión. Normalmente cuenta, además de los paneles, con filtros y comparativas; permitiendo responder a las preguntas ¿Qué pasó, porqué pasó y donde nos afectamos?

- **Estratégico:** Mira principalmente a una meta, teniendo un horizonte delimitado en meses, trimestres, semestres o años. Su granularidad suele ser más grande (meses o años) y muestra en sus paneles principalmente la comparación de un KPI con su objetivo para verificar qué tan bien o mal se está avanzando hasta esta métrica.

Ahora bien, observemos cómo se ordenaría cada KPI:

KPI	Nivel principal	Posible Nivel secundario	Herramienta
Energía	Operativo	Estratégico	Grafana
Yield	Analítico	Estratégico	PowerBi
Performance	Analítico	Estratégico	PowerBi
Ahorro	Estratégico		PowerBi
Disponibilidad	Estratégico	Operativo	PowerBi
Alarmas	Operativo	Analítico	Grafana
% de datos válidos	Operativo	Analítico	Grafana

Una nota sobre esta clasificación, se presentan dos opciones para cada kpi ya que según el objetivo de la lectura, estos KPIs pueden agregarse en un dashboard o al otro.

Para decidir la herramienta, se tomó en cuenta cual es el propósito principal de tomar cada KPI. Así, nos centramos en PowerBi para lo estratégico y analítico gracias a su capacidad de ofrecer más herramientas para la distribución de los datos, filtros y drill down; por otro lado, usamos Grafana para los casos operativos, aprovechando su capacidad de manejar mejor los datos en time series y de disparar alarmas.

D4. Grafana con PostgreSQL

Una consulta de series de tiempo debe devolver una columna de tipo timestamp ordenada de forma ascendente; para mostrarlo en un panel este debe ser de tipo time series. Las macros descritas tienen las siguientes funciones:

- *$__timeFilter(col)*: Actúa como un filtro dinámico en el where de nuestra consulta, que permite que la fecha del filtro se ajuste automáticamente a la del dashboard

- *$__timeGroup(col, '1h'):* Permite agrupar dinámicamente los datos mostrados en el dashboard en función del tiempo dado en el dashboard

La potencia promedio tiene la consulta se consigue con la consulta:

```
SELECT $__timeGroup(ts, '1h') AS time,
            avg(p_ac_kw) AS potencia_promedio
     FROM silver.lectura_5min
     WHERE $__timeFilter(ts)
     GROUP BY 1
     ORDER BY 1;
```

D5. Alertas

En Grafana Alerting se tienen las siguientes alertas:

- Las alert rules ejecutan una consulta, evalúan una condición y la sostienen por un tiempo
- Un contact point es el destino de la notificación generada por la alerta. Es decir, a qué servicio se reporta esta novedad

Una regla que podría sernos útil en este caso es:

“Si la potencia del inversor está por debajo del 10% de lo esperado con una irradiancia superior a 200 W/m^2 durante 15 minutos, notifique al equipo de mantenimiento por Slack”

Bloque 2 · Gobernanza de datos (data governance)

G1. Data governance, data management y data quality

El marco DAMA-DMBOK define la gobernanza de datos como el ejercicio de autoridad, control y toma de decisiones compartida sobre los activos de datos de una organización, diferenciándose de la gestión operativa diaria (DAMA International, 2017).

- El data governance (gobierno de datos) es el marco de políticas, roles y procesos que definen quién puede hacer qué con los datos, y bajo qué reglas.
- La data management (manejo de datos) es la ejecución de todas las actividades sobre los datos.
- La data quality (calidad de datos) es un área de esa gestión, y mide si los datos son aptos para su uso (exactitud, completitud, oportunidad).

G2.Data owner, data steward y data custodian. Relaciónenlos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).

- *Data owner:* es el responsable final del conjunto de datos, decide quién accede y para qué se usa. Sería el product owner del proyecto
- *Data steward:* cuida a diario la calidad, las definiciones y los metadatos. Aquí están el BI analyst y el data modeler.
- *Data custodian:* se encarga de la operación técnica, es decir, el almacenamiento, los respaldos y la seguridad. Sería el equivalente al data engineer

G3. Contrato de datos

El contrato de datos actúa como una interfaz formal entre quien produce el dato y quien lo consume, previniendo rupturas silenciosas cuando el esquema de origen cambia sin aviso (Shah, 2023).

Debe incluir:

- Esquema: nombres, tipos de datos y si se admiten nulos.
- Unidades: por ejemplo kW, °C, W/m^2.
- Frecuencia y latencia: cada cuánto llegan los datos y con qué retraso máximo.
- Responsable: persona o equipo de contacto.

- **Niveles de calidad**: por ejemplo completitud ≥ 98 %.
- **Versionado**: política de cambios y aviso previo.

Si la fuente cambia un campo sin avisar, la carga falla o inserta valores inválidos causando que los paneles se dañen o muestren data incorrecta. Para mitigarlo es necesario realizar validaciones en la ingesta.

G4. RLS

En Power BI, se crea un rol con un filtro DAX para que el usuario solo pueda ver los datos de su sitio. Para un responsable que solo ve esto , el filtro dinámico sobre la tabla de usuarios es:

```
'Usuarios'[correo] = USERPRINCIPALNAME()
```

La relación de esa tabla con Sitios limita las filas visibles

En Grafana el equivalente se hace con organizaciones (aíslan datos), equipos y permisos (Viewer/Editor/Admin) por carpeta o dashboard; para este caso no se filtra filas ya que para eso se usa un rol de PostgreSQL por fuente de datos o RLS en la base de datos.

G5.Ley 1581 de 2012

La Ley está definida en el marco legal colombiano:

"*Colombia regula el tratamiento de datos personales a través de la Ley 1581 de 2012, que define al titular, al responsable y al encargado del dato (Congreso de la República de Colombia, 2012).*"

Un dato personal es el que se asocia a una persona natural determinada y por medio de los cuales se puede identificar a dicha persona. La telemetría de la planta no es considerada como data personal pero si en algún momento el tablero muestra qué operador atendió cada alarma, la situación cambia. Para esto se requeriría autorización del titular, finalidad informada, medidas de seguridad y atención al habeas data. También habría que revisar si corresponde el registro de la base de datos ante la SIC.

Nuestro equipo se compromete a solo recolectar la data necesaria para lo operación lo necesario, usar pseudónimos para los usuarios, restringir el acceso y documentar la finalidad de cada dato.

Bloque 3 · Automatización de procesos ETL

E1.Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto. ¿Power Query es una herramienta ETL? Argumenten.

Antes de entender esta diferencia, debemos comprender a qué hacen referencia estos acrónimos, pues referencian los mismos procesos fundamentales:

1. **Extract** (Extracción) : Se realiza la extracción de los datos de diversas fuentes pero *sin procesar*. Esta fase solamente se encarga de obtener los datos "crudos" y en sus formatos originales, incluyendo en muchos casos metadata, origen de datos y fechas de consumo. Al ser data en su forma más básica, admite que se presenten duplicados o errores.

2. **Transform** (Transformación): En este punto ya se empiezan a aplicar las reglas del negocio a la data, buscando que esta sea confiable y que esté acondicionada para el procesamiento. En este proceso se busca eliminar datos duplicados, nulos innecesarios, hacer cálculos, realizar procesos de estandarización, entre otros. Luego de la transformación la data queda lista para su uso.

3. **Load** (Carga): Finalmente el flujo de trabajo carga la información al sistema destino en el cual será utilizado, realizando el proceso según el tipo de carga definido por el proceso

Estos procesos se diferencian principalmente en el orden en que ocurren sus pasos; al usar una ETL los datos son extraidos, procesados y luego cargados al sistema de almacenaje destinado para este proceso; en la ELT la carga se da *antes* de la transformación de los datos. La arquitectura Medallion (Bronze, Silver, Gold) se ajusta más a este último debido a la forma como fluyen los datos en este:

1. ¿Extract ocurre fuera del Medallion ya que solamente constituye el proceso de consulta desde las fuentes.

2. Load cae en la fase Bronze del proceso, otorgando la data cruda al proyecto

3. Finalmente Transform puede (según la necesidad del uso de la data) convertir la data en el paso Silver y Gold.

La mayor diferencia es que con una ETL el sistema de almacenaje recibe los datos ya transformados. En este caso, podemos decir que no hay una fase Bronze propiamente dicha en una ETL ya que no se tienen los datos crudos. En caso de detectarse un error en los datos crudos, se tiene que hacer una extracción nuevamente desde la fuente, en tanto que en una ELT se puede consultar la data extraída.

Para la última pregunta; debido a su funcionamiento en el cual extraemos la data de una fuente, luego usamos power query para limpiarla y finalmente la cargamos al modelo, podríamos encasillar que **sí es una herramienta ETL**. Lo principal que nos ayuda a distinguir entre ambas es que realmente nunca tenemos una versión cruda de los datos cargada en el modelo. Hacemos referencia a un datasource, el cual modificamos localmente y luego cargamos al modelo, pero esta fuente nunca hace parte realmente del proyecto en sí en su versión original.

E2.Full vs Incremental load

Full load e incremental load son estrategias de carga de datos desde la ETL al sistema de almacenamiento final, y que hacen parte importante de la arquitectura de dicho proceso pues nos permitirá garantizar la estabilidad y escalabilidad de nuestra infraestructura.

	Full Load	Incremental
Qué es	Es la carga completa de los datos al almacenamiento en una sola acción	Es la carga procedural de los datos
Cómo funciona	Se toman todos los datos recopilados y se cargan directamente al almacenamiento. Es la más directa y simple	Es un proceso más meticuloso. Solamente se cargan al almacenamiento aquellos datos que hayan cambiado con respecto a la fuente de datos inicial
Cuando usarlo	Es la mejor cuando la cantidad de datos no es demasiado grande o cuando se está inicializando el proyecto	Cuando la data tiene un ritmo de actualización relativamente rápido y/o cuando el dataset es muy grande. Esto permite optimizar el proceso de carga de datos a solo aquellos que de verdad necesitan ser actualizados

El proceso de *upsert* (combinación de los comandos UPDATE e INSERT) es una operación dentro de las bases de datos relacionales que permite o bien ingresar un dato como nuevo o actualizarlo si ya existe previamente. A partir de esta definición podemos inferir que dicha operación no debe generar registros duplicados que compartan una misma llave primaria en la base de datos; este concepto está atado a lo que llamamos *idempotencia*.

Una carga es idempotente cuando ejecutarla una o varias veces produce el mismo estado final, propiedad esencial para pipelines ETL confiables (Palma, 2022). Para la tabla `fact_energia_dia`, tenemos que se genera una llave primaria compuesta con las propiedades fecha_key y dispositivo_key. El proceso de inserción en SQL es por defecto idempotente para las llaves primarias, ya que estas no son duplicables en una misma tabla. Cada toma de datos nueva es un registro único y en el furtivo caso que la misma medida se

ingrese dos veces a la ETL, el proceso de upsert solo actualizará el registro inicial en lugar de duplicarlo. Si se quiere aún más seguro, se puede manejar únicamente un insert; si este proceso intenta agregar un registro con una PK duplicada, se generará un error de procesamiento, lo cual nos permitirá abortar esta operación.

E3. Cron vs Apache Airflow vs Power BI

Nombre proceso	Descripción	Qué automatiza	Cuando usarlo
CRON job	Es un planificador basado en tiempos que permite ejecutar comandos de forma automática a horas, fechas o intervalos específicos	Comandos individuales o scripts	Principalmente para la ejecución de tareas simples y aisladas, donde no haya dependencias entre los procesos
Apache Airflow	Plataforma de orquestación de flujos de trabajo, basado en Python que ejecuta grafos cíclicos basados en tareas con dependencias	Pipelines completos, incluyendo todos los procesos de un ETL, incluyendo la carga, procesos de reintento y validaciones entre ellas	Cuando lo que se busca agendar es un flujo de trabajo complejo, que requiere ejecutar código externo y tener un alto nivel de resiliencia en el proceso
Power Bi update	Función propia de Power Bi que consulta y actualiza los datasets de un reporte o dashboard	Se enfoca solo en refrescar los datos de los modelos de Power Bi	Cuando nuestro proceso está limitado a tener información actualizada en un reporte de Power Bi y cuando las fuentes estén listas.

E4. Query Folding

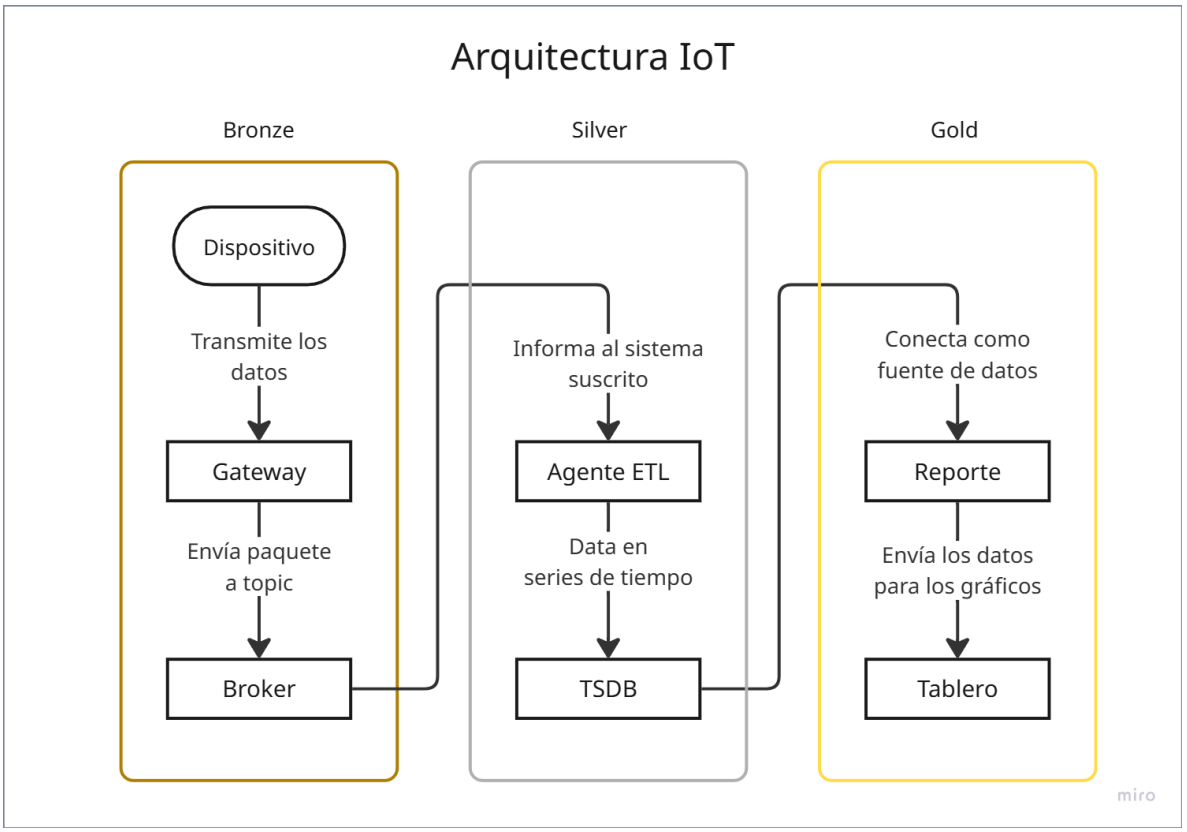
Antes de entender que es Query Folding debemos comprender qué es M. M, conocido también como Power Query M formula language es un lenguaje de programación funcional especialmente diseñado para permitir que Power Query pueda manipular datos de forma programática; este normalmente se genera cuando se hace la modificación de los datos en Power Query, pero también puede escribirse directamente en el editor avanzado.

El proceso de query folding es el proceso de optimización que de forma nativa hace power query sobre este código de M. En esencia, Power Query mira el código y determina qué bloques de este pueden ejecutarse de manera nativa en Power Bi, cuando encuentra referencias que no puede procesar o que son complejas de hacer nativamente, va a buscar diferir estos pasos a la fuente misma. Esto se hace ya que en la mayoría de los casos la fuente es mucho más eficiente en transformar la data que Power Query. Haciendo esto, el rendimiento de la consulta mejora mucho, ya que el sistema está recibiendo los datos ya procesados.

En nuestro ejemplo tenemos una base de datos de PostgreSQL con los datos de medición solar. Si necesitamos filtrar esta información por fecha, Power Query puede hacer un *plegado* de las solicitudes y enviar a la base de datos una sentencia DQL con el filtro de fecha correspondiente para que sea procesado por Postgres mismo. Si queremos confirmar que esto está sucediendo, podemos usar varias opciones, siendo las más directas consultar View Native Query en Power Query o consultando la tabla interna pg_stat_activity y pg_stat_statements para ver los procesos que han corrido de cara a la base de datos.

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas



I2. MQTT

El MQTT o *Message Queueing Telemetry Transport* es un protocolo diseñado para la comunicación máquina a máquina y para IoT. Este protocolo está basado en la arquitectura publish/subscribe, en la cual estas no se comunican directamente entre sí, sino con un servidor central al que llamamos **broker**. El funcionamiento de este tipo de arquitecturas es bastante útil ya que nos permite desacoplar el origen de los datos de los sistemas que los consumen, haciendo así que el sistema en sí pueda crecer de forma estable.

El funcionamiento en sí consiste en la interacción de los servicios con el broker. Dentro de este servidor se tiene uno o varios *temas* (topics) que son cadenas de texto que actúan como enrutadores para la información recibida; una máquina de origen que genera nueva información sobre su operación envía dichos datos al topic dentro del broker para que este luego se encargue de dirigir esta información a los *suscriptores*, los cuales no son sino otros sistemas que se encuentran conectados a este tema y que reciben la actualización de forma automática tan pronto el broker la libera. Un ejemplo de una jerarquía para un tema sería:

```
pascual_bravo/solar/parque_innovacion/cluster_solar_1/temperatura_superficie
```

En esta se tiene la siguiente estructura:

1. Raíz o fuente principal del proyecto
2. Localización
3. Tipo o identificador del dispositivo
4. Métrica

El protocolo MQTT ofrece 3 niveles de calidad de servicio:

- Qos 0: En el que el paquete de datos es entregado **máximo una vez** sin que se garantice su recepción.
- QoS 1: Garantiza que el paquete es recibido por lo menos una vez.
- QoS 2: Asegura que el paquete se entrega una y solo una vez (lo que incluye un handshake o verificación de conexión con el servicio destino)

Estos niveles entonces pueden ser usados según la criticidad de la data, intercambiando simpleza y velocidad por seguridad y confiabilidad.

I3. Monitoring Open Stack

Aquí hablamos de una stack tecnológico de soluciones open source para nuestro sistema de monitoreo solar (IoT), y las herramientas que están siendo descritas constituyen un componente para cada etapa del flujo de datos. Según esta descripción podemos construir el siguiente flujo:

1. Fuente de datos: Estos son los propios dispositivos que están generando los datos.
2. Mosquitto: Este es un broker MQTT tal como lo describimos anteriormente. Su mayor beneficio recae en que es ligero y optimizado para dispositivos con baja energía o redes poco estables. Su rol consiste en recibir los datos provenientes de los dispositivos y enviarlos a los sistemas de telemetría suscritos.
3. Telegraf: Es un agente ligero y de código abierto especializado en ejecutar pipelines de ETL/ELT, y que nos dará la posibilidad de transformar la información cruda proveniente de Mosquitto en datos enriquecidos que pueda almacenar en un warehouse para su uso posterior en una herramienta de dashboard.
4. TimeScaleDB / InfluxDB: Estas son TSDB (Bases de datos de series temporales) las cuales están especializadas en el manejo de datos con marcas de tiempo y políticas de retención. Su diferencia recae en que Timescale es un motor de base de datos propio, en tanto que Influx es una extensión de Postgres. Estas herramientas cumplen la función de almacenar la información proveniente de Telegraf ya transformada.

a. PostgreSQL puede tener varios usos aquí, según las decisiones de arquitectura tomadas al construir el pipeline de trabajo. Si se usa InfluxDB es prácticamente necesario usar Postgres ya que es una extensión de este; en cambio, al usar TimeScale, se puede usar postgres como una base de datos secundaria donde almacenemos la metadata del proyecto (como inventario de los dispositivos, locaciones, características). Este último rol puede igualmente cumplirse también en el primer caso, estando esto más allá del requerimiento mismo de usar pg para trabajar con Influx

5. Grafana: Finalmente, Grafana cumple el rol de herramienta de visualización de las métricas. Nos permite generar dashboards de control, gráficos y alertas, las cuales nos dan la posibilidad de hacer un seguimiento constante del estado de las máquinas.

I4. Batch processing vs Streaming

	Batch processing	Streaming
Qué es	Es el procesamiento de los datos en una ETL/ELT por paquetes. En este caso se envían los datos al pipeline cada cierto tiempo para ser procesados	Es el procesamiento continuo de la información. Un dato es procesado tan pronto entra al pipeline, con la menor latencia posible (limitada por la infraestructura misma)
Granularidad	Según la configuración, puede ser de minutos, horas, días, meses. Se consideran procesos de latencia alta	Mínima, de segundos.
Pros	Es más sencilla de implementar y más resiliente ante inestabilidades del sistema. También tiende a ser una solución más barata	Permite tener un control casi instantáneo de cambios en el origen de los datos, además de que la información en el sistema está prácticamente fresca en cada momento
Contras	Dependiendo de la necesidad, puede causar que se pierda información crítica del proceso. También, los lotes enviados para procesamiento son mayores, lo cual si no es calibrado correctamente puede causar una sobrecarga del pipeline	Requiere un sistema más robusto y complejo para funcionar correctamente, adicionalmente suele ser más caro de operar dado el mayor número de operaciones de escritura

Teniendo esto en cuenta, miremos la métrica propuesta; fact_energia_dia es una medida de la energía generada por un elemento en un día de trabajo, comparando esto contra la telemetría podemos decir:

- Se gana: Al agregar la data se puede ver el comportamiento promedio del elemento en todo el día, haciendo su seguimiento más sencillo en un reporte. Nos permite ver la métrica desde un nivel más alto, donde solo nos interesa el comportamiento general de este y no ir al detalle de las fluctuaciones en la generación del dispositivo. Adicionalmente, el procesamiento de esta información es más sencillo al permitir convertirlo en un solo punto de información en lugar de 288 si conservamos la granularidad original.
- Se pierde: Precisión en la medida. Si quisiéramos observar cómo trabaja un dispositivo en un momento específico de un día, detectar anomalías en el trabajo de la máquina, o identificar si hay momentos del día donde no hubieron datos registrados, nos sería imposible. También, si necesitamos comparar estos datos con otras métricas con una granularidad menor (como podría ser la temperatura del dispositivo) tampoco podríamos lograrlo correctamente.

Ahora bien, si queremos analizar la posibilidad de ambas granularidades podemos observar las herramientas más adecuadas para cada uno:

- Para una granularidad de cada 5 minutos necesitaríamos una herramienta enfocada en el control de series de tiempo para que soporte y controle correctamente la cantidad de registros de lectura con marcas de tiempo. En este caso, se podría usar Influx o Timescale
- Para la granularidad diaria las operaciones de escritura bajan bastante, permitiéndonos tener una herramienta más enfocada en el proceso de analítica, como BigQuery o Postgres.

Bloque 5 · Control de versiones

V1. Git y GitHub

Si bien están profundamente relacionados, *Git* y *GitHub* son dos conceptos que atienden a tecnologías que son fundamentales diferentes. Podemos encontrar una definición de ambos a continuación:

- **Git:** Es un *software de control de versiones* de código abierto y que es a grandes rasgos el estándar de la industria tecnológica para dicha función. Un software de control de versiones es un software que ofrece como principal recurso llevar un seguimiento detallado de los cambios que se hacen en un proyecto, guardando una "fotografía" del estado del mismo en un momento determinado y guardando esta captura con un identificativo propio. Esto le permite a los desarrolladores retroceder a versiones anteriores de su código para revertir cambios indeseados, guardar estados fijos del proyecto, e incluso trabajar en versiones paralelas del código en lo que llamamos "ramas" o "branches". Git se ejecuta principalmente en el entorno local de desarrollo, no necesitando de acceso a la red para la gran mayoría de sus funcionalidades.

- **GitHub:** Es una plataforma de desarrollo construída **sobre** el concepto y la funcionalidad de Git, pero que ofrece otras funcionalidades más allá de este. El producto principal de GitHub son los "repositorios", que son bases de código que se conectan con el sistema de control de Git, permitiendo guardar las versiones almacenadas del proyecto en la nube. Esto ofrece varias ventajas que complementan al ya de por sí poderoso funcionamiento de Git, ya que permite generar flujos de integración continua (proceso en el cual el proceso de prueba y despliegue del código se hace de forma automatizada), almacenar y disponibilidad sitios web basados en el código del repositorio y compartir este código con miles de usuarios con acceso a GitHub.

Como mencionamos anteriormente, Git y GitHub son prácticamente la dupla de software por defecto para el control de cambios en el código; sin embargo, para este último existen varios competidores que ofrecen alternativas (tanto técnicas como empresariales) a Github. Algunos de estos son GitLab y BitBucket. Los términos mencionados en el enunciado son sintaxis y procesos de git y github, y se definen así:

- Un **repository** (también llamado *repo* en el uso común) es una almacenamiento digital donde Git guarda el historial de versiones del proyecto al cual está adjunto. Cuando se corre el comando git init o se realiza un git clone, Git automáticamente crea esta carpeta invisible en la raíz del proyecto y guarda cada fotografía del código en esta. La copia de esta carpeta es la que se sube a GitHub, permitiéndole a este sistema hacer seguimiento del proyecto de la misma manera que lo hace el sistema local.

- Un **commit** es lo que venimos llamando *fotografía* en las explicaciones anteriores. Es una versión autocontenida, estática e inmutable del código en un momento preciso del tiempo. Cuando se *hace un commit* del código, estamos guardando una versión inalterable del estado del código en el repositorio.

- Una **branch** o **rama** es una lista enlazada de las versiones o *commits* del repositorio en el que se está trabajando. Cuando se inicia un proyecto, automáticamente se crea una rama principal (usualmente llamada main o master) en donde se empiezan a guardar las versiones del código; cada vez que enviamos un commit a una rama este se guarda como la versión más reciente de este; a la versión previa del código se le llama *parent* y a la nueva *children*.

  El poder de las ramas es que podemos crear una línea temporal paralela que parta de un commit de la rama main pero que no hace commit directamente a esta. Esto traduce en que podemos modificar el código cuanto queramos sin que esto repercuta directamente en main (que suele ser la versión del código que se encuentra en producción).

- Un **pull request** es un proceso por medio del cual se busca unificar dos ramas del repositorio, comúnmente se hace desde una rama de trabajo hacia main. El proceso de pull request busca que el proceso de unificación (lo cual se denomina como *merge*) sea lo más controlado posible, muchas veces requiriendo que el código en la rama entrante pase por pruebas funcionales, se garanticen controles de seguridad, se solucionen conflictos y que se tenga una correcta documentación de cambios antes de reintegrarse al repositorio principal, lo cual busca asegurar la integridad de main.

Un archivo .pbix es binario ¿Qué problema genera en Git y qué resuelve el formato Power BI Project (.pbip)?

La situación con el archivo .pbix es un problema bastante particular y está relacionado con la descripción dada originalmente de Git. Los archivos .pbix, tal como lo menciona el enunciado, son binarios, es decir que son una combinación de 1's y 0's en octetos que son interpretados por un programa específico (PowerBi en este caso). El problema recae en que cada cambio hecho al reporte en PowerBi por mínimo que sea genera un binario diferente; aún peor, al ser este binario sólo interpretable por PowerBi en sí, GitHub no tiene ningún medio de identificar a qué cambio corresponde esta nueva versión. Por ejemplo, si cambiamos el color de un gráfico y exportamos este reporte a pbix, al guardarlo en GitHub se nos mostraría un cambio completo en el archivo sin máyor detalle a qué se debió este.

Los archivos .pbip (PowerBi Project) solucionan este problema al compilar el reporte en un *proyecto* exportable. Este archivo muestra la versión actual del trabajo hecho en BI como una serie de archivos JSON y TMDL, los cuales muestran en un esquema estructurado lo que contiene el reporte, lo que hace mucho más sencillo el proceso de controlar qué modificaciones sufrió este archivo contra su versión anterior.

¿Cómo se versiona un dashboard de Grafana?

Según la necesidad puntual, Grafana puede ofrecer dos opciones para realizar el manejo de versiones en su plataforma.

La primera es una forma nativa. Cada vez que se guarda una versión del dashboard, Grafana guarda una versión del mismo en un sistema interno similar a git; este proceso es similar al versionamiento que algunas plataformas como Google Docs o Google Sheets usan para permitir a los usuarios restaurar versiones antiguas de un documento. Al igual que estos, Grafana le permite a los usuarios revisar los cambios que dicha versión introdujo, compararlo con la versión actual e incluso restaurar dicha versión si es necesario.

En segundo lugar, Grafana permite usar la funcionalidad de *Git Sync* la cual permite sincronizar los recursos con un repositorio basado en Git. Esto lo hace al permitir guardar los archivos en formato de JSON (similar a lo observado anteriormente con los archivos .pbip) y cargarlos a una plataforma compatible con git. Sin embargo este proceso va más allá, ya que permite tener una integración bidireccional entre la base de datos de Grafana, el repositorio y la interfaz de usuario. El sistema central de Git Sync permite que los cambios realizados en la UI sean cargados directamente al repositorio, pero también que los cambios del repositorio viajen directamente a la ui. Esto es posible gracias a que git sync en cada proceso de sincronización actualiza la base de datos que alimenta a los dashboards, permitiendo así que se descentralice el procesamiento de los datos y la estructuración de los reportes a solo los sistemas propietarios de Grafana.

4. Parte B · Práctica guiada: los mismos datos en dos herramientas

Paso 1 · Bronze: datos crudos

Se ejecuta el script compartido en el trabajo:

```python
import csv, math, os, random
from datetime import datetime, timedelta

inicio = datetime(2026, 10, 5, 0, 0)

os.makedirs("data/bronze", exist_ok = True)

with open( "data/bronze/telemetria.csv", "w", newline= "", encoding= "utf-8") as
f:
   w = csv.writer(f)

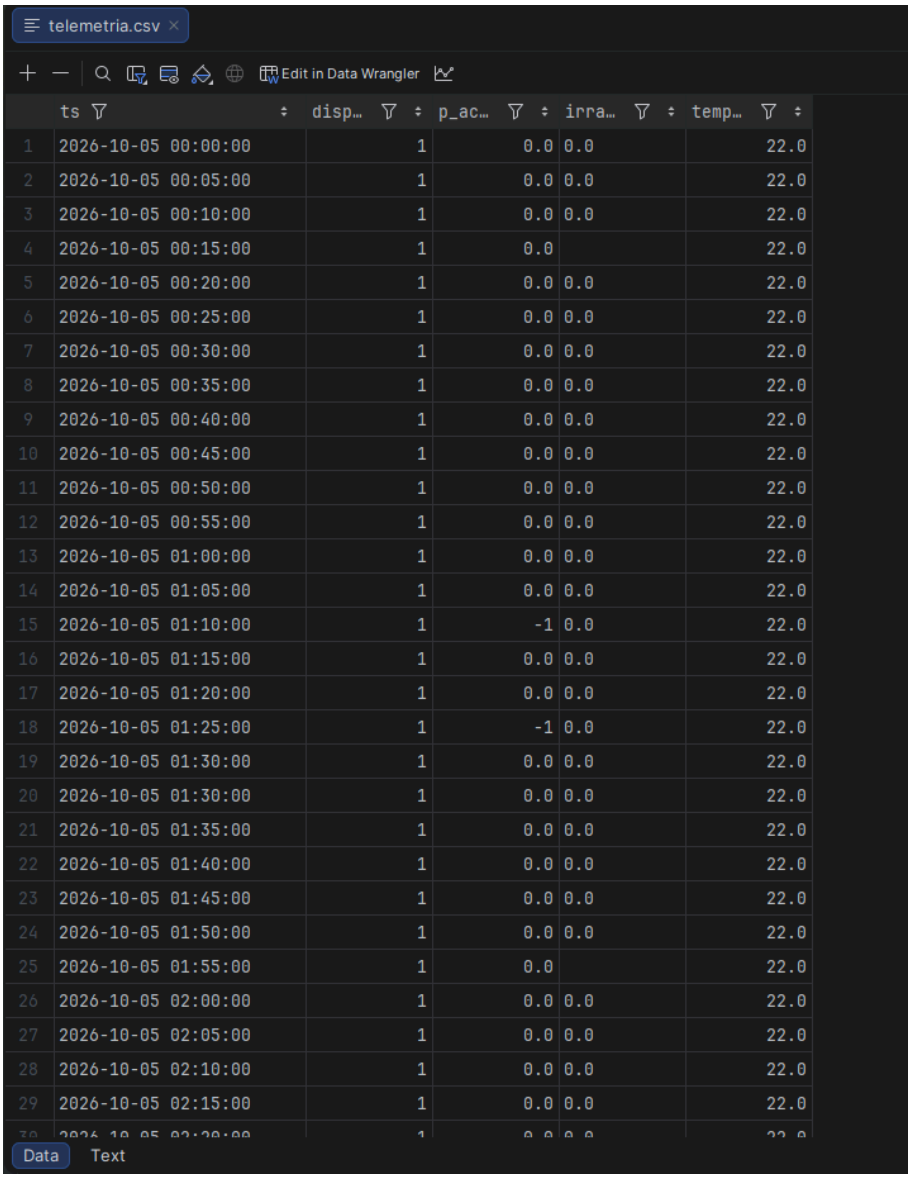
w.writerow(["ts","dispositivo_id","p_ac_kw","irradiancia_wm2","temp_modulo_c"])

   for i in range(3 * 288):  # 3 dias, cada 5 min
       ts = inicio + timedelta(minutes=5 * i)

       sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
       irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
       p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
       fila = [ts.isoformat(sep= " "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
       r = random.random()
       if r < 0.02:
           fila[2] = -1  # anomalia: potencia negativa
       elif r < 0.04:
           fila[3] = " "  # anomalia: dato faltante
       w.writerow(fila)
       if r > 0.98:
            w.writerow(fila)  # anomalia: fila duplicada
```

simulador.py

Con este se crea el archivo con la data cruda



	ts	disp…	p_ac…	irra…	temp…
1	2026-10-05 00:00:00	1	0.0	0.0	22.0
2	2026-10-05 00:05:00	1	0.0	0.0	22.0
3	2026-10-05 00:10:00	1	0.0	0.0	22.0
4	2026-10-05 00:15:00	1	0.0		22.0
5	2026-10-05 00:20:00	1	0.0	0.0	22.0
6	2026-10-05 00:25:00	1	0.0	0.0	22.0
7	2026-10-05 00:30:00	1	0.0	0.0	22.0
8	2026-10-05 00:35:00	1	0.0	0.0	22.0
9	2026-10-05 00:40:00	1	0.0	0.0	22.0
10	2026-10-05 00:45:00	1	0.0	0.0	22.0
11	2026-10-05 00:50:00	1	0.0	0.0	22.0
12	2026-10-05 00:55:00	1	0.0	0.0	22.0
13	2026-10-05 01:00:00	1	0.0	0.0	22.0
14	2026-10-05 01:05:00	1	0.0	0.0	22.0
15	2026-10-05 01:10:00	1	-1	0.0	22.0
16	2026-10-05 01:15:00	1	0.0	0.0	22.0
17	2026-10-05 01:20:00	1	0.0	0.0	22.0
18	2026-10-05 01:25:00	1	-1	0.0	22.0
19	2026-10-05 01:30:00	1	0.0	0.0	22.0
20	2026-10-05 01:30:00	1	0.0	0.0	22.0
21	2026-10-05 01:35:00	1	0.0	0.0	22.0
22	2026-10-05 01:40:00	1	0.0	0.0	22.0
23	2026-10-05 01:45:00	1	0.0	0.0	22.0
24	2026-10-05 01:50:00	1	0.0	0.0	22.0
25	2026-10-05 01:55:00	1	0.0		22.0
26	2026-10-05 02:00:00	1	0.0	0.0	22.0
27	2026-10-05 02:05:00	1	0.0	0.0	22.0
28	2026-10-05 02:10:00	1	0.0	0.0	22.0
29	2026-10-05 02:15:00	1	0.0	0.0	22.0

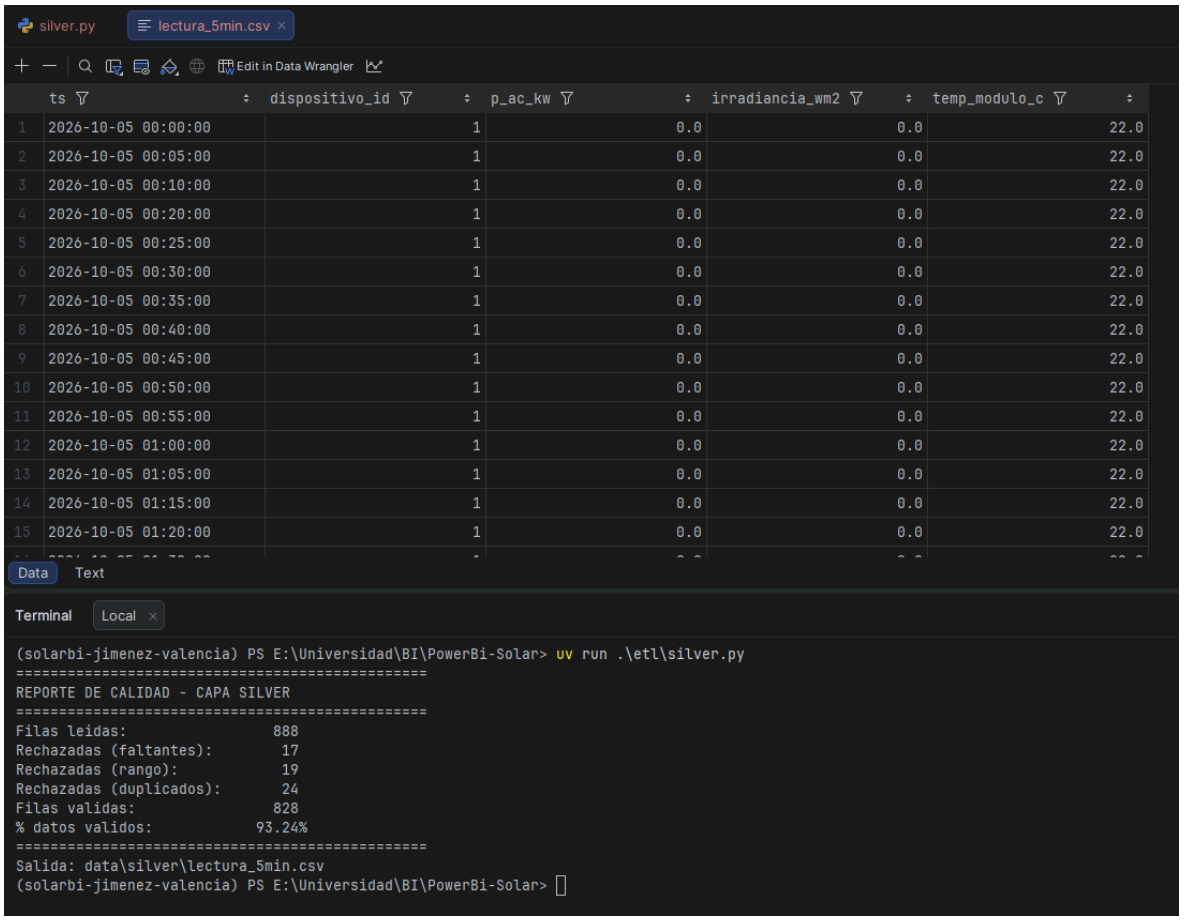
Archivo telemetría.csv

Paso 2 · Silver: reglas de calidad

Realizamos la limpieza de los datos siguiendo las siguientes reglas de calidad:

1. Faltantes elimina las filas del reporte donde el valor de alguna de las columnas críticas (potencia del inversor, irradiancia o temperatura) esté vacía.
2. Rango elimina las filas donde la magnitud de la métrica en alguno de los campos reporta un valor fuera de lo establecido por los márgenes.
3. Duplicados realiza un dedup a los datos

Con este proceso se crea el archivo lectura_5min.csv



ts	dispositivo_id	p_ac_kw	irradiancia_wm2	temp_modulo_c
2026-10-05 00:00:00	1	0.0	0.0	22.0
2026-10-05 00:05:00	1	0.0	0.0	22.0
2026-10-05 00:10:00	1	0.0	0.0	22.0
2026-10-05 00:20:00	1	0.0	0.0	22.0
2026-10-05 00:25:00	1	0.0	0.0	22.0
2026-10-05 00:30:00	1	0.0	0.0	22.0
2026-10-05 00:35:00	1	0.0	0.0	22.0
2026-10-05 00:40:00	1	0.0	0.0	22.0
2026-10-05 00:45:00	1	0.0	0.0	22.0
2026-10-05 00:50:00	1	0.0	0.0	22.0
2026-10-05 00:55:00	1	0.0	0.0	22.0
2026-10-05 01:00:00	1	0.0	0.0	22.0
2026-10-05 01:05:00	1	0.0	0.0	22.0
2026-10-05 01:15:00	1	0.0	0.0	22.0
2026-10-05 01:20:00	1	0.0	0.0	22.0

Archivo `lectura_5min.csv`

Paso 3 · Gold: carga en PostgreSQL

Se crea el reporte unificado, normalizando la energía producida a kWh en el archivo fact_energia_dia:

```
agregado = (
    df.groupby(["fecha_key", "dispositivo_id"], as_index=False)
    .agg(
        energia_kwh=("p_ac_kw", lambda s: round(s.sum() * (5 / 60), 3)),
        lecturas=("p_ac_kw", "count"),
    )
    .rename(columns={"dispositivo_id": "dispositivo_key"})
)
agregado["pct_datos_validos"] = (
    agregado["lecturas"] / LECTURAS_POR_DIA * 100
).round(2)
return agregado[
    ["fecha_key", "dispositivo_key", "energia_kwh", "pct_datos_validos"]
]
```

Muestra `gold.py`

Punto de entrada en run_etl:

```python
def correr(nombre: str) -> None:  2 usages  Miguel Valencia
    print(f">>> Ejecutando {nombre}")
    resultado = subprocess.run(
        args: [sys.executable, str(ETL_DIR / nombre)],
        check=False,
    )
    if resultado.returncode != 0:
        sys.exit(f"Fallo {nombre} con codigo {resultado.returncode}")


def main() -> None:  1 usage  Miguel Valencia
    print(">>> PASO 1/2: SILVER")
    correr("silver.py")
    print()
    print(">>> PASO 2/2: GOLD")
    correr("gold.py")
    print()
    print("Pipeline ETL completado.")
```

run_etl.py

NOTA: Asumimos como parte de nuestra arquitectura que los datos crudos (bronze) serán generados y alimentados al sistema desde un elemento ajeno a este código (Posiblemente Mosquitto), por lo cual run_etl sólo ejecuta las capas silver y gold

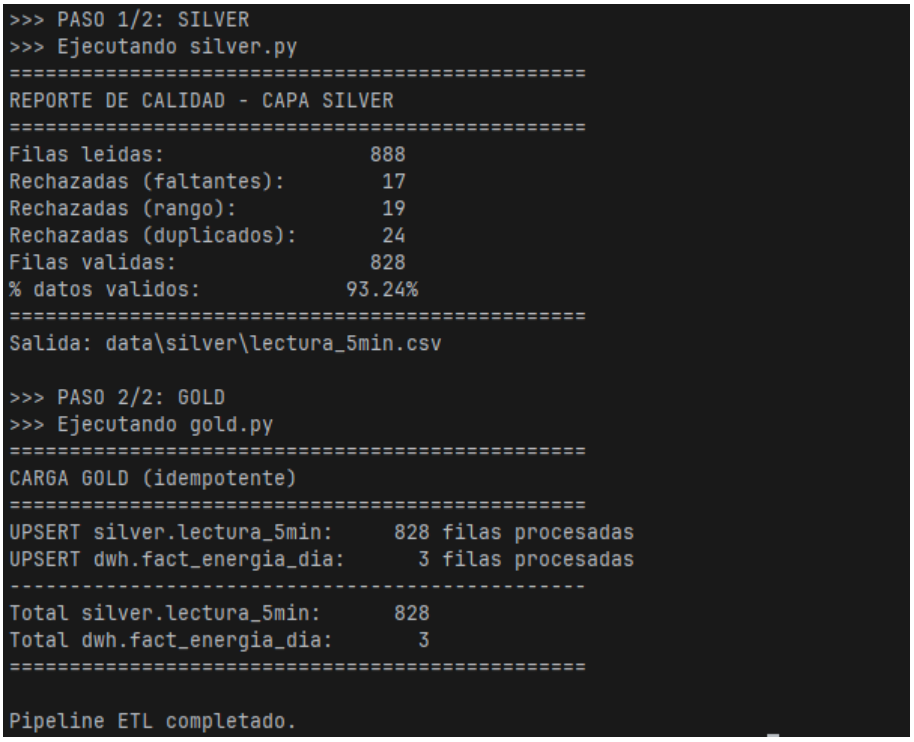
Para programar esta ejecución, dependiendo del sistema que estamos usando cambiará un poco.

Para linux / mac:

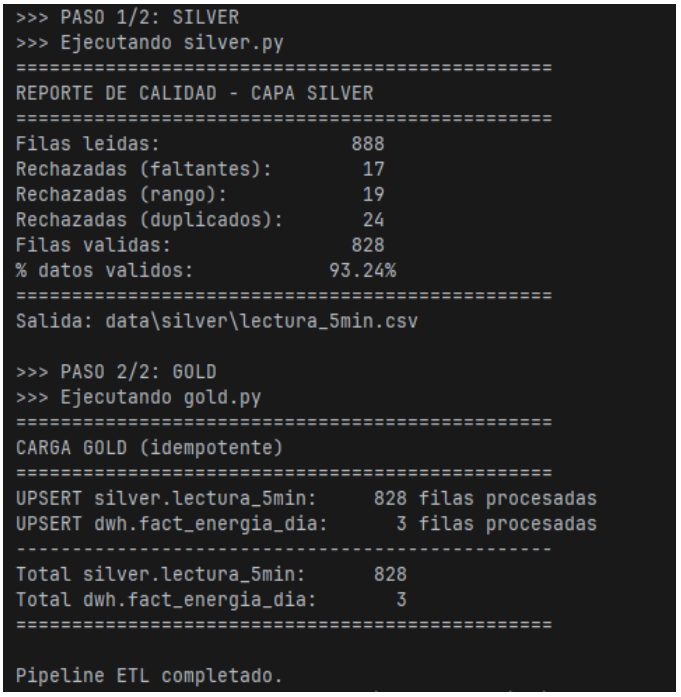
```
0 0 * * * cd /ruta/al/repo && uv run python etl/run_etl.py
```

Para Windows usaremos schtasks:

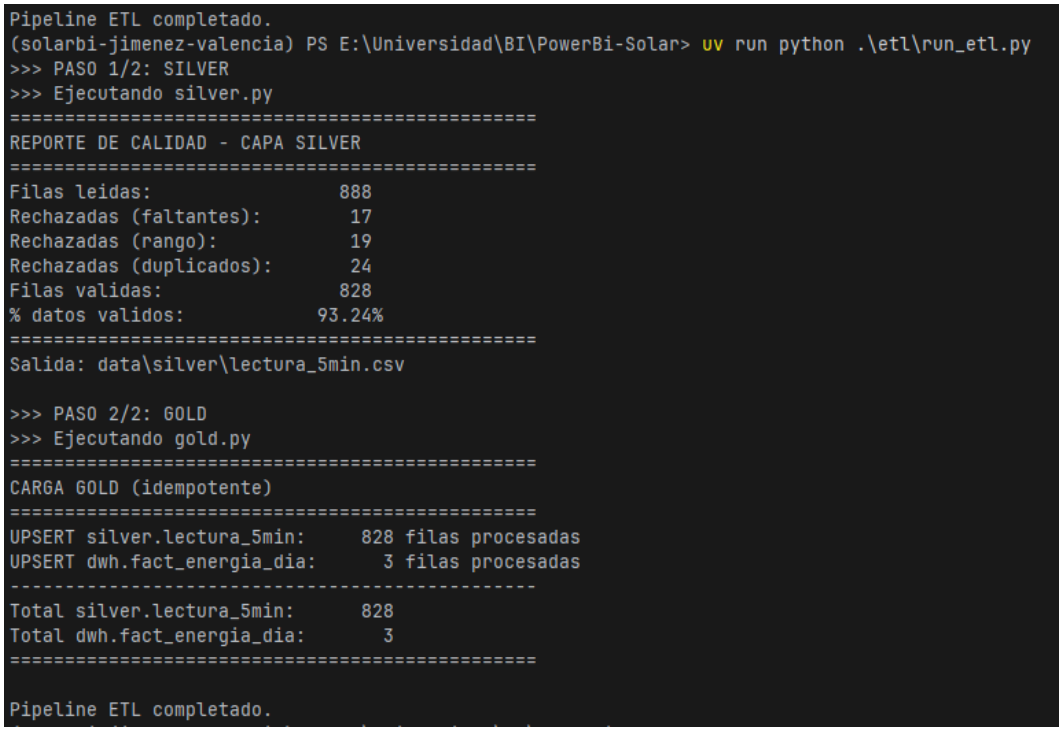
```
schtasks /Create /SC DAILY /TN "SolarBI ETL diario" ^
    /TR "cmd.exe /c cd /d E:\ruta\al\repo && uv run python etl\run_etl.py" ^
  /ST 00:00
```



Ejecución 1



Ejecucion 2



Ejecución 3

Podemos corroborar que los 3 pasos son idempotentes

Paso 4 · Power BI

Se crean las medidas DAX:

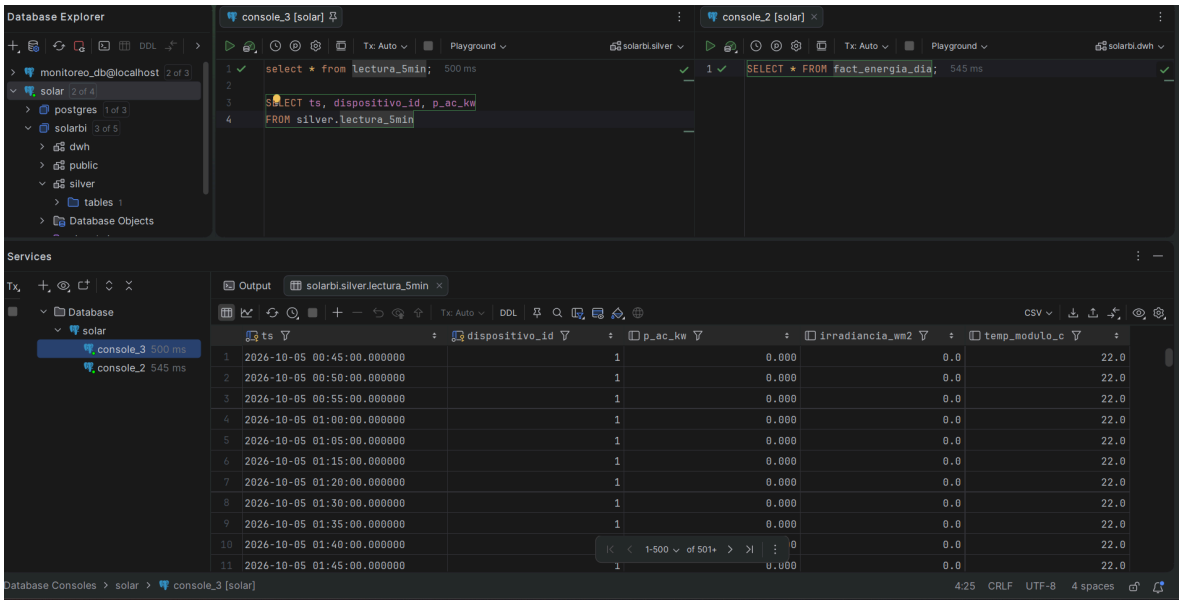
Suma de la energía en un día

```
Energia_total_kwh = SUM('dwh fact_energia_dia'[energia_kwh])
```

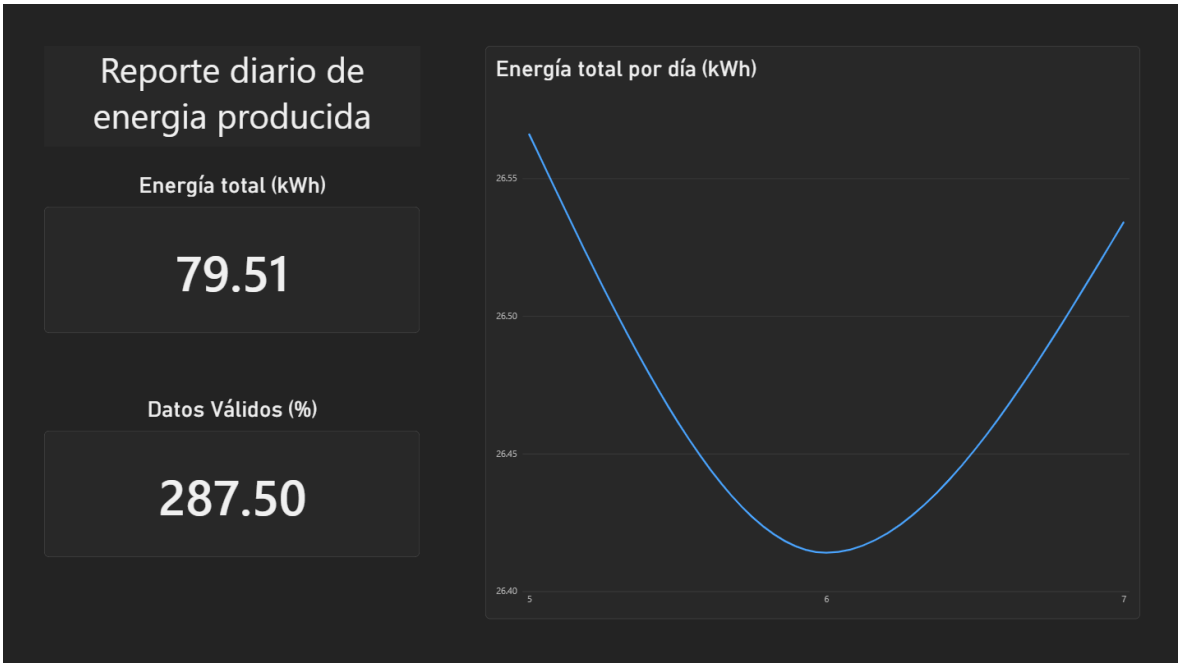
Promedio de datos válidos expresados en porcentaje:

```
Prom_pct_datos_validos = AVERAGE('dwh fact_energia_dia'[pct_datos_validos])
```

Luego se usa el conector nativo para unirse a la base de datos Postgres local



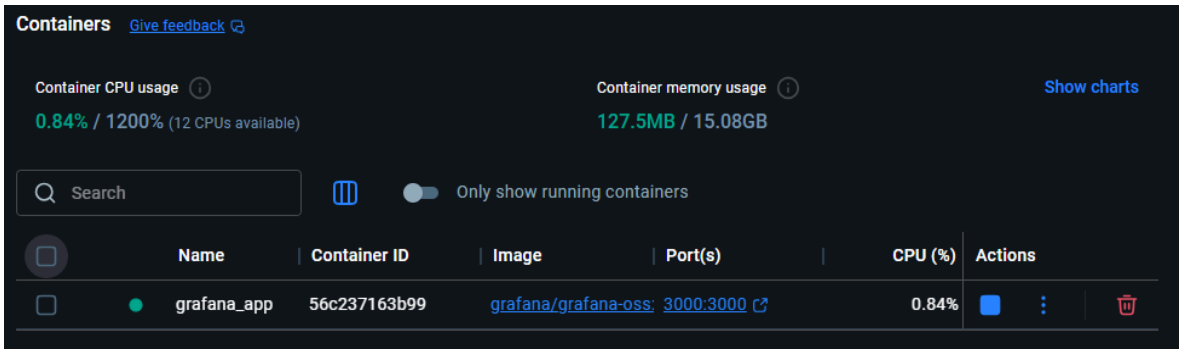
Base de datos local



Dashboard en PowerBi

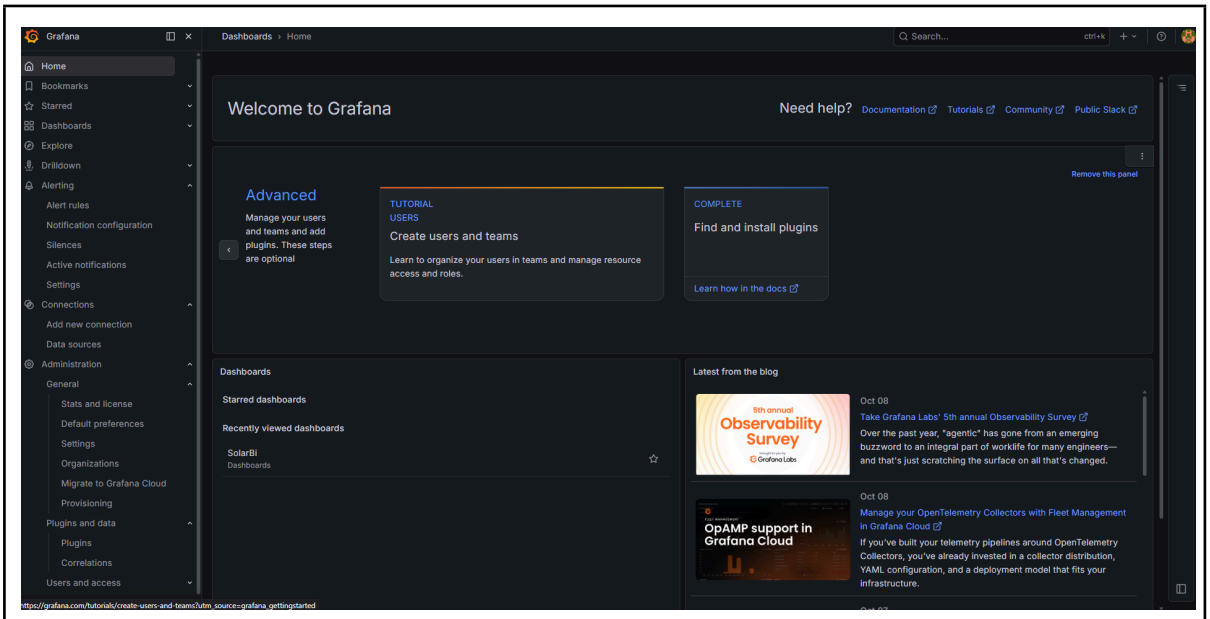
Paso 5 · Grafana

Se realizó la instalación local de grafana OSS por medio de Docker



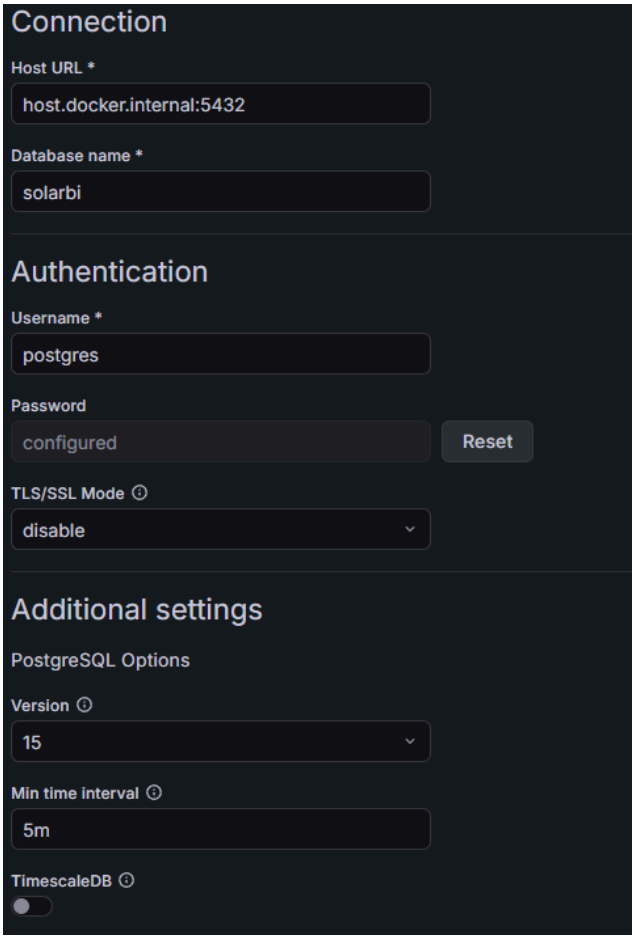
Docker desktop

Luego corremos Grafana en el puerto 3000



Grafana Dashboard

Añadimos Postgres como data source (lo configuramos como un túnel desde docker ya que la DB no está en un contenedor)



Config Postgres



Dashboard Grafana



Detalle serie de tiempo



Detalle panel stat




```
{
  "apiVersion": "dashboard.grafana.app/v2",
  "kind": "Dashboard",
  "metadata": {
    "name": "adx7qln",
    "namespace": "default",
    "uid": "cdb01c4c-dbcc-4fbf-9a86-75225a647b26",
    "resourceVersion": "1791757284243996",
    "generation": 6,
    "creationTimestamp": "2026-10-11T21:29:50Z",
    "labels": {
      "grafana.app/deprecatedInternalID": "1711455080681472"
    },
    "annotations": {
      "grafana.app/createdBy": "user:fg0xiqwcenw1sb",
      "grafana.app/folder": "",
      "grafana.app/saved-from-ui": "Grafana v13.0.2 (3fcdbc5a)",
      "grafana.app/updatedBy": "user:fg0xiqwcenw1sb",
      "grafana.app/updatedTimestamp": "2026-10-11T22:21:24Z"
    }
  },
  "spec": {
    "annotations": [
      {
        "kind": "AnnotationQuery",
```


Detalle JSON de Grafana

Paso 6 Reflexiones

Miguel

Ambas herramientas tienen grandes puntos fuertes y fue bastante ilustrativo usarlas. En lo personal, siento que PowerBi es un poco más compleja de usar, pero al mismo tiempo es más completa y tiene un ecosistema más rico que Grafana; la configuración de los archivos DAX es muy similar a lo que usualmente manejamos de SQL, pero es mucho más sencillo manejar SQL directamente, como se hace en Grafana. Siento que el punto fuerte de PowerBi son los dashboard ejecutivos y administrativos, enfocados a transmitir información para directivos y stakeholders, en tanto que Grafana, con su funcionalidad de generar alertas, crear dashboard más simples y conectarse con sistemas de comunicación para entregar dichas alarmas está mucho mejor enfocado a los operadores de planta.

5. Parte C · Repositorio en GitHub

Se puede encontrar el repositorio a continuación:

- https://github.com/Koi-No-Yokan/PowerBi-Solar/commits/main/

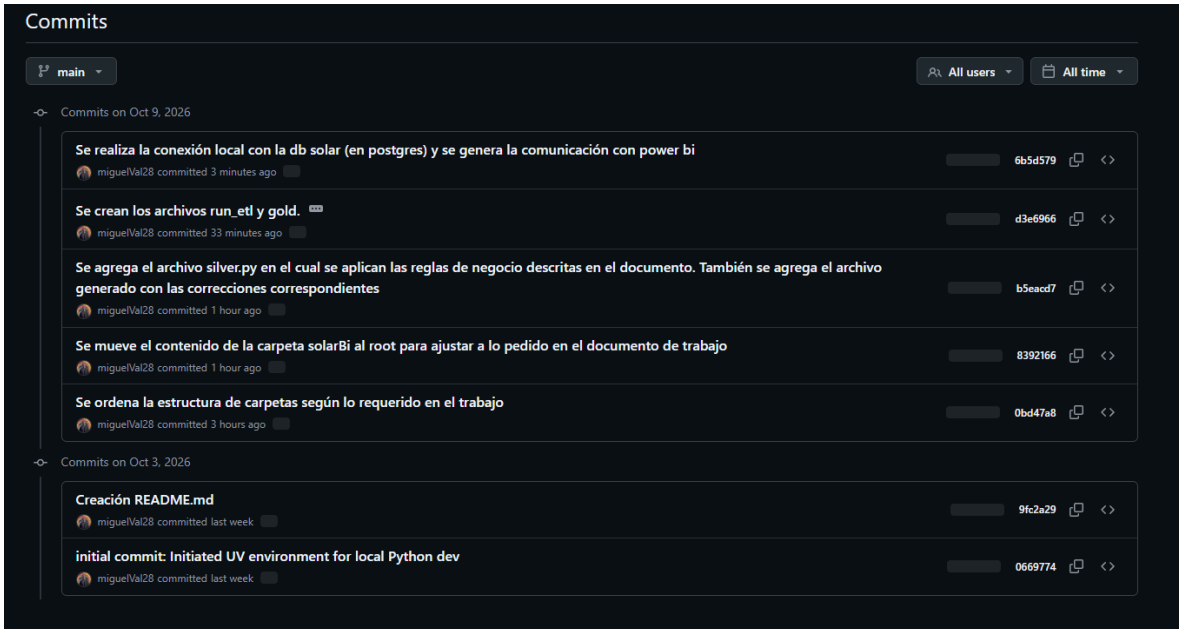
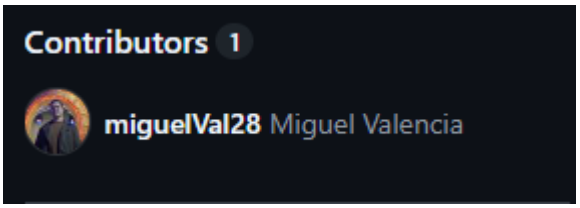


Imagen de los commits en el repositorio



Contribuyentes en el repositorio

Referencias

Apache Software Foundation. (s. f.). Apache Airflow documentation. https://airflow.apache.org/docs/

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/en/v2

Congreso de la República de Colombia. (2012, 17 de octubre). Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial N.° 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

DAMA International. (2017). DAMA-DMBOK: Data management body of knowledge (2nd ed.). Technics Publications.

Eclipse Foundation. (s. f.). Eclipse Mosquitto: An open source MQTT broker. https://mosquitto.org/

GitHub. (s. f.). About pull requests. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

Grafana Labs. (s. f.). Grafana OSS documentation. https://grafana.com/docs/grafana/latest/

Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.

Microsoft. (s. f.-a). Descripción de la evaluación de consultas y plegado de consultas en Power Query. Microsoft Learn. https://learn.microsoft.com/es-es/power-query/power-query-folding

Microsoft. (s. f.-b). Guía de inicio rápido: Introducción a DAX. Microsoftft.com/es-es/dax/dax-overview

Microsoft. (s. f.-c). Seguridad de nivel de fila (RLS) con Power BI. Microrosoft.com/es-es/power-bi/enterprise/service-admin-rls

OASIS. (2019). MQTT version 5.0: OASIS standard. OASIS Open. https://docs./mqtt-v5.0.html

Palma, D. (2022, 15 de marzo). Core data engineering: Idempotency. Medium.neer-things/core-data-engineering-idempotency-ddc5f3f6c6b7

PostgreSQL Global Development Group. (s. f.). PostgreSQL 17 documentation.ocs/current/

The pandas development team. (s. f.). pandas documentation. https://pandas